ACTIA®
ENGINEERING SERVICES

Embedded Team Academy

Advanced C++ training

Prepared and Presented by
Mahdi SAKKA

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

Agenda

Module 1: C++ Basics

Module 2: C++ Object Oriented Programming

Module 3: C++ Standard Library

Module 4: C++ Advanced Programming

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

Agenda

Module 4: C++ Object Oriented Programming

- Templates
- Type traits
- Metaprogramming
- Variadic templates
- Introduction to design patterns
- Introduction to coding rules

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

Templates

- Templates are a powerfull tools for C++ that allows generic programming
- It consists in creating functions or classes with generic types (one or many type)
- The compiler will compile the created templates only if there are called by filling out the generic types with concrete types
- Compiler can sometimes perform deduction of the type during the template function / calls call, but it is better to always explicitly provide the type
- A template is compiled only when it is used by a caller object for specific type (a template class / function containing an error doesn't generate it unless it is called with given type)

Templates

- Function template: allows creating generic function. Make sure that the types to be applied satisfy the implementation of generic functions so that no compile error will be faced. For example, make sure that a member function of template type is existing. See example below:

```
template<typename T, typename U>
T function_name(const U& u)
{
    T t;
    t = u.process();

    return t;
}
```

U should have method 'process()'

T should be the return of process function or compatible type

- Function template should always defined in header files («.h»)

Templates

- Function template: the template types can be used as function parameter, return type or simply in the function body

```
template<typename T>
T min(T& x, T& y)
{
    return (x < y ? x : y);
}
```

Return and parameters use the template type, we don't use «const T» because T may be const itself

```
template<typename T>
int processFunction(int in)
{
    T t(in);

    return t.process();
}
```

Template type is used into the function implementation, the instantiation supposes that T has a constructor with an int as parameter

- It is noted that the instantiation of template function is compile time operation

Templates

Function template: template specialization means that we create a specialized version of function template for a specific type. It supposes then that the generic template exists and another one is created with same name and already specified type:

```
template<typename T, typename U>
T function(const U& u)
{
    std::cout << "Called template" << std::endl;
    T t;
    t = u.process();

    return t;
}
// template specialization for function
template<>
int function(const A& a)
{
    std::cout << "Called template speicalization" << std::endl;
    return a.process2();
}
```

Template specialization is called when ‘function’ is called with A type as parameter

```
A a(2);
// A is auto deduced (cannot deduct return type)
int x = function<int>(a);
```

Templates

- Function template: non-type template parameters: means that we can put a template parameter as non generic type, possible values:
  - Integral type (e.g. int, char, long, etc...)
  - Enumeration
  - Pointer or reference

```
template<int N>
void print()
{
    std::cout << "N for this call is " << N << std::endl;
}
int main()
{
    print<5>();
}
```

```
template<int (*fn)()>
void print()
{
    std::cout << "Return for this call is " << fn() << std::endl;
}
int test() {return 5;}

int main()
{
    print<test>();
}
```

Templates

- Class templates: like function templates, class templates define a type that will be applicable for the whole class (visible for all members variables and functions)
- Every class template called with a type represents a new type. For example, Test<A> has a different type from Test<B>.
- Template class should be declared and defined in header files as well

```
template<typename T>
class Test
{
public:
    Test(T& t);
};
```

Declaration

```
template<typename T>
Test<T>::Test(T& t)
{

}
```

Definition

Templates

- Class templates: template specialization for particular type is possible for classes as well
- The class has to be defined with specific type

```
template<typename T, typename U>
class A
{
public:
    A(T& x);

    A(T&& x);

    virtual U process();
protected:
    T m_x;
};
```

Specialization

```
template<>
class A<char, char>
{
public:
    A(char& x) : m_x(x)
    {
        std::cout << "A template specialization char (& constructor)" << std::endl;
    }

    A(char&& x)
    {
        std::cout << "A template specialization char (&& constructor)" << std::endl;
    }

    virtual char process()
    {
        std::cout << "A template specialization char process" << std::endl;
        return 'X';
    }
protected:
    char m_x;
};
```

Templates

- Class templates: partial specialization means that we fix a type of class template for a well defined type and leave another one as template type
- It has to be done for the totality of the class
- If only one member function to be specialized, we may create a new class that inherit from specialized parent class
- See example provided for demo

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

Type traits

- Type traits defines a set of functions very useful in template programming that allows identifying the type of parameters of templates
- They are defined in header file <type_traits>
- They are generally used with static_assert to verify that the template type is compatible with the template implementation
- Generally used at compile time to make some verifications on template types properties

Metaprogramming

- In C++, metaprogramming refers to the use of templates to perform calculation at compile time
- It will consist then in generating code during compilation
- We will use for this purpose:
  - Templates resolution
  - Enumerations
  - Ternary operator: (cond ? if_true : if_false;)

Metaprogramming

- For example, let's see the famous factorial function: f(x) = f(x-1) * x with f(0) = 1

```cpp
template<int N>
struct Fact
{
    enum
    {
        value = Fact<N-1>::value * N
    };
};

template<>
struct Fact<0>
{
    enum
    {
        value = 1
    };
};
```

```cpp
int main()
{
    constexpr int x = Fact<10>::value;
    std::cout << "Precompiled value for factorial of 10 = " << x << std::endl;
    return 0;
}
```

The value of x is calculated by generating code during compile time

- Limitation: many compilers allow a maximum number of recursion depth

Variadic templates

- Ellipsis represents a variable arguments list (noted by three points '...')
- In C style, va_list is used to manage ellipsis. This is prohibited in modern C++
- A variadic template is a template that accepts one or more template parameter
- Syntax: '*template<typename... pack_name>*'
- Programmer can add also template parameters other than variadic parameter pack:

  '*template<typename T, typename... pack_name>*'

Variadic templates

- For template functions, the declaration will be:

```
template<typename... Args>
void function_name(Args... args);
```

- For template classes / structures:

```
template<typename... Args>
class Class_Name
{
};
```

- For both of examples above, simple types can be added to the template

```
template<typename T, typename... Args>
void function_name(Args... args, T t);
```

Variadic templates

- For classes methods definition, the syntax should be:

```
template<typename T, typename... Args>
void Class_name<T, Args...>::member_name(Args... args, T t);
```

- Variadic classes and functions are used to pass as many as needed types to the corresponding structure and of course at compile time (cannot perform type definition during runtime)
- Generally, an effcient way to use variadic templates are recursive call of variadic templated classes or functions (see examples)
- std::tuple can be used as well for parameter pack management (see examples)

Perfect forwarding

- "Forwarding" is the process where one function forwards its parameter to another function
- When it is perfect, the function should receive the same object passed from the function that does the forwarding i.e. perfect forwarding means we do not just forward objects, we also forward their salient properties, whether they are lvalues or rvalues, const or volatile.
- The perfect forwarding is to be considered in template programming
- Let's see the problem of forwarding in relation with templates in steps...

☐ Provisoire ☐ Public ☐ Interne ☐ Confidentiel ☐ Secret

ACTIA®

Perfect forwarding

- Consider the following: the compile error is due to ‘a’ is an lvalue reference so it won’t
- Possible solutions:
  - Add const to ‘Arg’, but ‘a’ then cannot be changed anymore in the function
  - Overload by adding a new version with const lvalue reference

```
template <typename T,typename Arg>
T create(Arg& a){
  return T(a);
}

int main(){
  // Lvalues
  int five = 5;
  int myFive= create<int>(five);
  std::cout << "myFive: " << myFive << std::endl;
  // Rvalues
  int myFive2= create<int>(5); // COMPILE ERROR
  std::cout << "myFive2: " << myFive2 << std::endl;
  return 0;
}
```

Perfect forwarding

- Now it compiles successfully, the solution works !
- However, if we have more parameters, we have to create more and more overloads … (not so good)
- Also, we cannot move the parameter const reference even it accepts a rvalue at function call
- The solution is to use the «universal reference» for template argument

```
template <typename T,typename Arg>
T create(Arg& a){
    return T(a);
}
template <typename T,typename Arg>
T create(const Arg& a){
    return T(a);
}

int main(){
    // Lvalues
    int five=5;
    int myFive= create<int>(five);
    std::cout << "myFive: " << myFive << std::endl;
    // Rvalues
    int myFive2= create<int>(5);
    std::cout << "myFive2: " << myFive2 << std::endl;
    return 0;
}
```

Perfect forwarding

- «Universal reference» encodes information about the lvalueness and rvalueness of the arguments that are passed to them. Let's see example:
- We use std::forward to acheive the perfect forwarding
- It keeps the property of passed argument (rvalue or lv

```
template <typename T,typename Arg>
T create(Arg&& a){
  return T(std::forward<Arg>(a));
}

int main(){
  // Lvalues
  int five=5;
  int myFive= create<int>(five);
  std::cout << "myFive: " << myFive << std::endl;

  // Rvalues
  int myFive2= create<int>(5);
  std::cout << "myFive2: " << myFive2 << std::endl;
  return 0;
}
```

Perfect forwarding

- Extention to variadic templates:
- It allows having a generic function that accepts all types of parameters even user defined types

```
template <typename T, typename ... Args>
T create(Args&& ... args){
    return T(std::forward<Args>(args)...);
}

struct MyStruct{
    MyStruct(int i, double d, std::string s){
        std::cout << "MyStruct parameters " << i << " "
                  << d << " " << s << std::endl;
    }
};

MyStruct myStr= create<MyStruct>(2011,3.14,str4);
```

Introduction to Design Patterns

- Design patterns are typical solution that has been created to resolve commonly occuring problems in software design (not only for C++)
- Design patterns are a toolkit of tried and tested solutions to common problems in software design. Even if you never encounter these problems, knowing patterns is still useful because it teaches you how to solve all sorts of problems using principles of object-oriented design.
- There are a multitude of design patterns, the list is long and we cannot detail all of them during this training, but we will try to see some examples

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

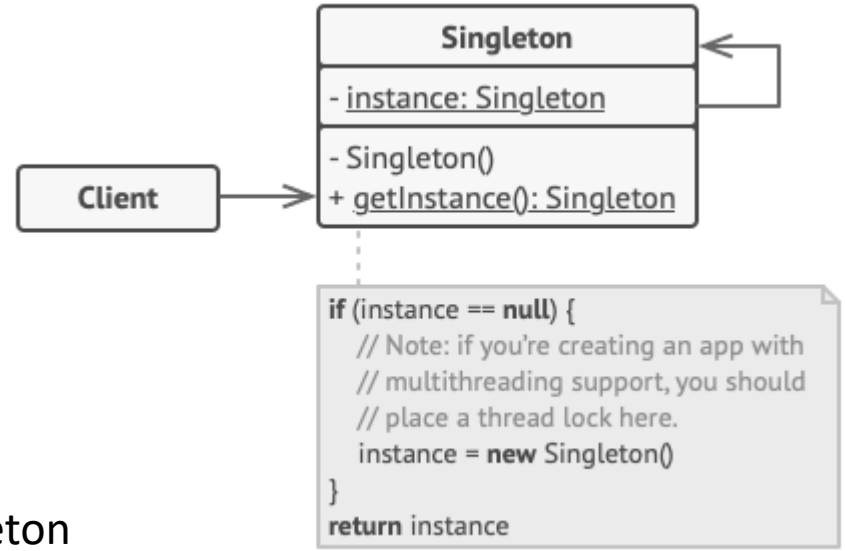
Introduction to Design Patterns

- Here is a list of some famous desing patterns:
  - Singleton
  - Strategy
  - State
  - Factory
  - Adapter
  - Bridge
  - Observer
  - Decorator
  - Visitor
  - Memento
  - Template Method
  - Command

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

Introduction to Design Patterns: Singleton

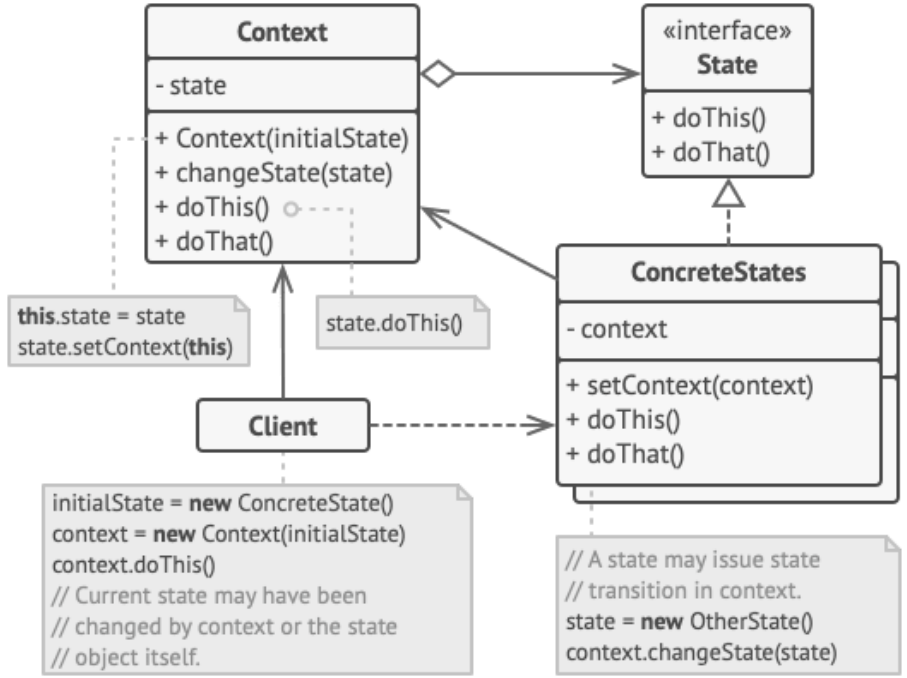
- Singleton design pattern ensures that a class has only one instance, while providing access to this instance
- Used for objects that must be unique like a database for example



source: https://refactoring.guru/design-patterns/singleton

Introduction to Design Patterns: State

- State design pattern is used to manage object that has internal state change. The typical implementation is used for Finite State Machine FSM
- The idea is to create a context class that stores the initial state and delegate the state related behavior to that states (transition to other states as well)
- All concrete states should inherit from a base interface state
- See examples for typical implementation



Introduction to Design Patterns: Strategy

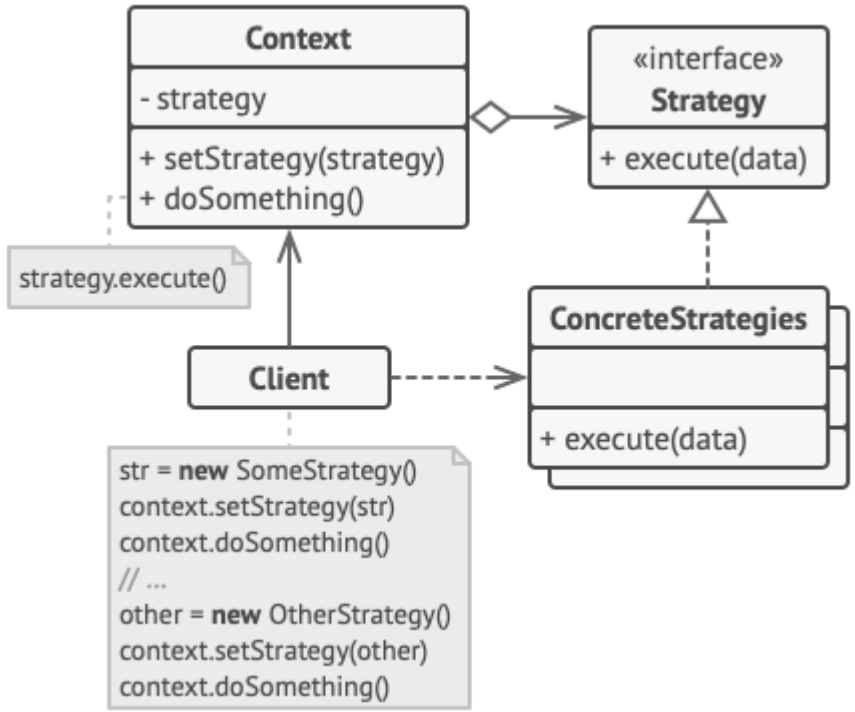
- Strategy design pattern allows separating the class (client) that does the processing that could be done in different ways from the algorithm of these ways (called strategies). The user of the client is responsible of the selection of the strategy or behavior
- An example of use: a class Travel will be used to manage trip information. A strategy is the mean of transport. The user will have to set the travel mean at each time according to cost, region, ...
- Strategies then will be Bicycle, bus, train, airplane...

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Introduction to Design Patterns: Strategy

- Strategies or behaviors can be changed without changing the context source code



Introduction to Coding rules

- Coding rules include:
  - Coding guidelines related to the use of programming language facilities
  - Coding style to have a tidy and well formed source code with well defined naming convension for classes, functions, members, ...
- Developers within a project should be aware of the used coding rules and follow them

Introduction to Coding rules

- Coding guidelines or standards try to avoid errors that are caused by code poorly written i.e. using programming language facilities in a way that make them vulnerable to errors or make the meaning not clear.
- There are multitude of coding guidelines
- MISRA coding rules is one of the most famous rules
- Also, Autosar C++ 14 coding rules, for example, is spreadly used in automotive domain
- These rules or guidelines are checked using static code analyzers (cppcheck, codesonar, ...)

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

Introduction to Coding rules: AUTOSAR C++14

- AUTOSAR C++14 are coding guidelines «*for the usage of the C++14 language, in the safety-related and critical environments, as an update of MISRA C++:2008, based on other leading coding standards and the research/analysis done by AUTOSAR*»
- Link for the document of AUTOSAR C++14 coding rules here

Introduction to Coding rules: Code Style

- Code style is also a set of rules that fix the form of the source code e.g.
  - Tabulation or space for code source indentation
  - Position of the braces {}
  - Spaces before and after signs

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®
ENGINEERING SERVICES

AN ENTREPRENEURIAL SPIRIT

TO MOVE FORWARD TOGETHER

ACTIA Engineering Services

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®